AI CLI Profile Manager 使用手册

适用版本：`0.3.19`（源码与安装目标；发布、安装和 Live 证据须分别核对） 适用系统：Windows 11 x64、PowerShell 7 命令入口：`aicli`

AI CLI Profile Manager 是原生 Codex CLI、Claude Code 与 Open Interpreter 的启动和运维层。它负责保存 Profile、隔离 Provider 环境、体检和连通测试；聊天界面、历史会话、工具执行和上游账号仍由各 CLI 自己负责。

> 本手册是产品操作的唯一事实源。三套上游 CLI 的会话内命令见《Codex、Claude Code 与 Open Interpreter CLI 中文手册》。

1. 五分钟开始

1.1 安装前提

- Windows 11 x64。
- PowerShell 7，命令为 `pwsh`。
- 至少安装一个目标 CLI：Codex、Claude Code 或当前官方 Rust Open Interpreter。
- 使用云端 API 的 Profile 需要对应 Provider 的 Key；官方订阅 Profile 使用上游自己的登录。

1.2 安装本工具

`0.3.19` 是当前源码与安装目标；源码提交、GitHub Release、已安装 payload 和 Live 回执不是同一层证据。从源码工作树安装时直接使用本节后面的 `scripts\Install.ps1`。使用正式发布版时，从 GitHub Releases 下载同一版本的 ZIP 和 `.sha256.json`。下面的命令会先核对发布清单，再解除这个已核对 ZIP 的 Internet 阻止标记；不需要也不应该全局放宽 ExecutionPolicy：

```
$version = '<从 Releases 页面选择的已发布版本>'
$base = "https://github.com/wlyaaaaa/ai-cli-profile-manager/releases/download/v$version"
$download = Join-Path $HOME "Downloads\ai-cli-profile-manager-$version"
New-Item -ItemType Directory -Force -Path $download | Out-Null

$zip = Join-Path $download "ai-cli-profile-manager-$version-win-x64.zip"
$hashJson = Join-Path $download "ai-cli-profile-manager-$version.sha256.json"
Invoke-WebRequest "$base/$(Split-Path $zip -Leaf)" -OutFile $zip
Invoke-WebRequest "$base/$(Split-Path $hashJson -Leaf)" -OutFile $hashJson

$expected = (Get-Content -LiteralPath $hashJson -Raw | ConvertFrom-Json).sha256
$actual = (Get-FileHash -LiteralPath $zip -Algorithm SHA256).Hash
if ($actual -ne $expected) { throw 'SHA256 不匹配，拒绝安装。' }

Unblock-File -LiteralPath $zip
$extract = Join-Path $download 'package'
Expand-Archive -LiteralPath $zip -DestinationPath $extract -Force
Set-Location $extract
pwsh -File .\scripts\Install.ps1
```

如果已经手动下载并核对文件，从解压后的目录执行安装器即可：

```
pwsh -File .\scripts\Install.ps1
```

目标版本已经存在时，安装器默认拒绝覆盖。确认替换同一版本时使用：

```
pwsh -File .\scripts\Install.ps1 -Force
```

安装器会：

1. 把模块安装到当前用户的 PowerShell 模块目录，不要求管理员权限。
2. 创建 `%LOCALAPPDATA%\aicli\bin\aicli.cmd` 与 `aicli.ps1` 垫片，并尝试加入用户 `PATH`。
3. 尝试在当前用户 PowerShell 7 配置中加入模块自动导入块。
4. 先验证临时候选版本，再替换目标版本；失败时恢复原版本。
5. 从 `0.3.10` 之前的版本升级时，先预检再隔离可证明由 AICLI 管理的 Qwen3.7 旧入口，同时保留新 06-08 exact Profile。

退役迁移不会把 Qwen3.7 自动改投 Qwen3.8，也不读取、复制、移动或删除 SecretRef/密钥。身份、marker、body/state 哈希与内容寻址都闭合的旧用户 Profile、Codex TOML/catalog 和旧模块版本会移入可恢复目录 `%LOCALAPPDATA%\AiCliProfileManager\retirement\qwen37-v1`。任何未知、用户改写或 reparse 项都会在任何安装变更前失败关闭；先审计该路径，不要盲目删除。

新开一个 PowerShell 7 窗口后验证：

```
Get-Command aicli
aicli version
```

当前窗口还找不到 `aicli` 时：

```
$env:Path = "$env:LOCALAPPDATA\aicli\bin;$env:Path"
Import-Module AiCliProfileManager
aicli version
```

开发者也可以不安装，直接在仓库根目录运行：

```
pwsh -File .\bin\aicli.ps1 version
pwsh -File .\bin\aicli.ps1 doctor
```

1.3 第一次设置

```
aicli setup
```

`setup` 会检查 Windows、PowerShell、目标 CLI、Profile 和 Ollama，并让你选择要配置的方向。静态检查不会让模型生成内容；配置第三方云端 Profile 时才会要求无回显输入 API Key。

1.4 开始第一段对话

在项目根目录运行：

```
aicli start codex-official
# 或
aicli start claude-official
```

也可以显式指定含空格或中文的目录：

```
aicli start codex-official --project "C:\Work\Demo App"
aicli start claude-official --project "D:\项目\演示"
```

命令会立即启动一个新的原生 CLI 进程。父终端的 Provider 环境变量不会被永久修改。

1.5 工作区日常用法（精确 Codex Profile）

在**受信任的项目根目录**用一条命令进入所需的精确 Codex CLI：

```
cd E:\你的项目
# 例：cd C:\Work\my-project

aicli start codex-qwen3-7-max-paygo --project (Get-Location)
# 或 Qwen3.8 Max
aicli start codex-qwen3-8-max-paygo --project (Get-Location)
# 或 DeepSeek Flash / V4 Pro (CLI-only)
aicli start codex-deepseek-flash --project (Get-Location)
aicli start codex-deepseek-v4-pro --project (Get-Location)
```

说明：

- `--project` 必须是已经确认可信的工作区；Profile 自己封闭 Provider、模型、Responses wire、MAX 与 SecretRef。
- Qwen3.7 只恢复 `codex-qwen3-7-max-paygo` → `qwen3.7-max-2026-06-08`；通用 alias、其他快照、Plus、旧 Profile、导入和原生 `--model` / fallback 仍失败关闭。
- DeepSeek Codex 当前保留下列两个隔离身份：

模型身份	Profile
`deepseek-flash`（官方自动升级 Flash ID）	`codex-deepseek-flash`（桌面 + CLI）
`deepseek-v4-pro` / `DeepSeek-V4-Pro-0813`	`codex-deepseek-v4-pro`

1.6 安装或升级 aicli 之后必须用新会话

`Install.ps1 -Force` 或手动覆盖模块目录后，**已经打开的 PowerShell 窗口仍可能加载旧模块。**会出现「别人已修好、你这边还在报 `engine` / 旧行为」的假象。

任选其一：

```
# 推荐：关掉终端，新开 pwsh 再运行 aicli
# 或在当前窗口强制重载：
Remove-Module AiCliProfileManager -Force -ErrorAction SilentlyContinue
Import-Module AiCliProfileManager -Force
aicli version
```

2. 先理解三个对象

对象	保存什么	是否包含秘密
Provider 模板	随产品发布的引擎、端点、模型和能力声明	否
用户 Profile	选择的模板、地域、模型、项目偏好和秘密引用	只保存引用
启动计划	本次可执行文件、工作目录、参数和子进程环境	运行时短暂持有必要秘密

`aicli start` 每次根据模板和用户 Profile 重新生成启动计划，因此更新或重新安装上游 CLI 后，不需要把 Key 写进全局环境变量。Profile 切换只对**新进程**生效，不能在已经打开的会话中途热切换 Provider。

`aicli run` 是供上层程序使用的 Codex harness，不等于交互式 `start`：所有当前和未来 Codex 模型固定使用原生 `danger-full-access` 与 `approvalPolicy=never`。规则按 `engine=codex` 生效，因此未来新增 Profile 自动继承；显式请求 `read-only` / `workspace-write` 会在模型调用前失败关闭。运行还必须回读 actual model、modelProvider 与 `dangerFullAccess` 权限身份，并拒绝任何模型重路由。只把受信任工作区交给该入口。

同一通用规则默认注册受管 `public_web_search`：仅访问固定 `https://cn.bing.com/search` RSS，拒绝重定向、Cookie、模型指定 endpoint/Header/Key，并把结果标作不可信公共文本。Windows 系统代理只用于固定 HTTPS 出口且不提供默认凭据。事件和回执仅记录搜索生命周期、provider 与次数，不记录查询/结果正文；本次不需要网络时使用 `--no-web-search`，权限仍是 `danger-full-access`。

`0.3.11` 起，`aicli run start` 为每个任务建立 root-owned 持久 run；任务正文仍只从 stdin 输入，不写入恢复状态。调用方用 `run status` 读取状态，用 `run resume` 继续被中断的 exact thread，用 `run abort` 协作中止；`start/resume --background` 会立即返回 run id。恢复前后必须一致回读 thread/session、workspace、Profile 指纹、model/provider、requested/effective effort 和权限；任一项变化都会失败关闭。不能证明 exact resume 时，只能在新 workspace 建立新 attempt 全量重跑，不得合并旧 partial。

非交互恢复控制面：

```
$task | aicli run start codex-ollama-main --stdin --json --project C:\Work\Project --background
aicli run status <run-id> --json
aicli run resume <run-id> --json --background
aicli run abort <run-id> --json
```

3. Profile 管理

3.1 命令总览

```
aicli profile list
aicli profile list --available
aicli profile list --available --json
aicli profile show codex-deepseek-flash
aicli profile configure codex-deepseek-flash
aicli profile configure codex-deepseek-v4-pro --reuse-secret-from codex-deepseek-flash
aicli profile set-default codex-official
aicli profile remove qwen-work
```

命令	作用	生效时间
`profile list`	显示可直接启动或已经配置的 Profile	立即
`profile list --available`	显示全部公开模板和当前状态	立即

命令	作用	生效时间
`profile show`	脱敏显示 Provider、端点、模型、密钥是否存在和数据去向	立即
`profile configure`	保存用户 Profile；三家受管厂商只存密码中心引用，其他本地密钥接法保持兼容	下次启动
`profile set-default`	设置无参数入口优先使用的 Profile	下次无参数运行
`profile remove`	删除用户实例；不删除内置模板	立即

删除型命令会先确认。使用 `--yes` 可跳过交互确认。删除最后一个引用某密钥的用户 Profile 时，也会删除本工具保存的对应 DPAPI 密钥文件；不会删除上游官方登录。

3.2 当前公开 Profile

引擎	场景	Profile
Codex	ChatGPT/OpenAI 官方登录	`codex-official`
Codex	Qwen3.7 Max 06-08 Workspace 按量 Responses	`codex-qwen3-7-max-paygo`
Codex	Qwen3.8 Max Workspace 按量 Responses	`codex-qwen3-8-max-paygo`
Codex	GLM-5.3 / GLM-5.3-Flash 中国区 Responses	`codex-glm-5-3` 、 `codex-glm-5-3-flash`
Codex	DeepSeek Flash / V4 Pro（CLI-only） Responses	`codex-deepseek-flash` 、 `codex-deepseek-v4-pro`
Codex	本机精确 main / 35B 交叉 / Qwen3.8-27B	`codex-ollama-main` 、 `codex-ollama-review` 、 `codex-ollama-qwen3-8-27b`
OpenCode	本机精确 main / Qwen3.8-27B	`opencode-ollama-main` 、 `opencode-ollama-qwen3-8-27b`
Claude Code	Claude 官方登录	`claude-official`
Claude Code	本机 Ollama	`claude-ollama`
Claude Code	自定义 Anthropic Messages 兼容端点	`claude-custom`
Open Interpreter	本机 Ollama	`oi-ollama`

`codex-deepseek-flash` 使用官方自动升级 ID `deepseek-flash` ， `codex-deepseek-v4-pro` 使用固定 `deepseek-v4-pro` / `DeepSeek-V4-Pro-0813` 。两者均走官方 Codex integration 的 Responses wire、1M context、 `low` / `high` / `max` ，用户默认 `max` ，并拒绝模型、Provider 与 fallback 覆盖；V4 Pro 仅为 CLI-only。旧 `codex-deepseek` / `deepseek-v4-flash` 、Claude Code 与 Open Interpreter 的 DeepSeek 模板均已退役。动态变化以 DeepSeek Change Log 为准。

两个 Qwen exact Profile 都只接受北京百炼 Workspace 按量 Responses endpoint，并固定 983616 context、95% 有效窗口和 885254 token（最大上下文 90%）自动压缩阈值。 `codex-qwen3-7-max-paygo` 只固定 `qwen3.7-max-2026-06-08` ； `codex-qwen3-8-max-paygo` 只固定 `qwen3.8-max-0902` 。用户 `max` 均映射 effective= `xhigh` 。Qwen3.7 的通用 alias、其他快照、preview、Plus，以及 Qwen3.8 的可变 alias、preview、Token Plan 与模型覆盖继续失败关闭。

配置过 `codex-qwen3-8-max-paygo` 后，Codex 桌面桥接会在每次启动时请求原生引擎刷新在线模型目录，并只接受带 `etag` 、抓取时间和客户端版本的原生在线缓存，再把 `Qwen3.8 Max 0902` 合并进模型选择器。刷新会做三次有界重试；仍不可验证时交还原生 Codex。 `debug models` 内部即使静默退回可能含旧模型的内置目录，其输出也不会进入菜单。密钥由受管 Profile 的加密副本按需交给 Codex 的 command-backed auth（命令取令牌认证）通道，不写入基础 `config.toml` 、模型目录或全局环境变量。上游官方模型仍动态刷新；关闭或移除这条云 Profile 不会固定、过滤或覆盖 OpenAI 模型。

`codex-deepseek-flash` 使用 DeepSeek 官方 Responses 端点 `https://api.deepseek.com` 和模型 ID `deepseek-flash` 。桌面菜单显示“DeepSeek Flash”，不写死 V4.1 版本号；官方会把这个 Flash ID 指向当前 Flash 版本。它支持文本、图像、工

具调用和 1M 上下文。旧 `deepseek-v4-flash` 已退役并拒绝启动；`deepseek-v4-pro` 仍是官方合法 ID，仅保留在独立 CLI Profile，不显示在桌面菜单。

`codex-glm-5-3` 与 `codex-glm-5-3-flash` 使用智谱中国区 `https://open.bigmodel.cn/api/v1` Responses 端点，分别固定模型 ID `glm-5.3` 与 `glm-5.3-flash`，不使用 alias（动态别名）或 fallback（后备模型）。两者均声明 1048576 token 上下文、95% 有效窗口、943718 token（最大上下文 90%）自动压缩线和 `low` / `high` / `max` 推理档位；Flash 在 Codex 桌面目录中声明文本与图像输入。根任务和子代理默认使用简体中文，思考摘要与进度重点解释与你的目标有关的发现、原因和影响；需要调查或分步处理时，开始前会主动说明准备解决的问题，过程中在重要发现、阶段进展或调整方向时继续解释，不把说明全部留到结束。不固定行数，用你关心的目的和影响解释内部操作，也不为增加篇幅凑话。最终答复按问题需要展开，让必要细节足够清楚。两项 GLM 模型使用 Codex 的延迟工具搜索，插件工具按需加载；智谱 Responses 返回原始 reasoning content 时，桌面桥会把同一内容映射为可见思考。两个 Profile 共享 Password Center 的 `zhipu-glm-api` 凭据来源，但各自保留独立 Provider 与模型目录，切换不同服务的历史任务时仍需新建任务。

同一套“开始前说明、多步过程中讲清重要发现和影响、最终答复保留必要解释”的表达方式也用于 AICLI 受管的本地 Qwen、云端 Qwen 和 DeepSeek Codex 模型。它不会改变你在菜单中选择的模型、模型速度、上下文长度或 OpenAI 官方模型；OpenAI 模型仍由 Codex 自己动态更新。其他模型的实际表达效果会随模型本身而不同，当前已由实际使用验收的是 GLM。 Qwen、GLM 和 DeepSeek 的 Key 按厂商保存在密码中心，Profile 只引用 `passwordcenter:<厂商>`。CLI 在每次使用时通过一次性本机管道接收值，再注入实际客户端子进程环境；新版 Codex Desktop 的 command-backed auth 经桌面身份入口取值，原生认证管道不会把 Key 写入模型目录或普通配置。旧桌面桥按名字加载 0.3.19 模块时，其 `Get-AiCliSecret` 调用会转到同一厂商的普通终端交付，不再执行 `aicli-glm-codex-profile-import`，也不创建 DPAPI 副本。正在运行的旧桥可以继续取 key，不必为这次模块兼容改动重发旧桥。

只配置引用不证明密码库已有可用 key。`profile show`、列表和 `doctor` 明确显示“引用已配置，密钥未验证”；真实调用成功才构成可用证据。桌面 auth.command 刷新间隔为 0，每轮可能重新解开密码库，验收应跑两轮并记录耗时。

这三家厂商的本机交付允许明文返回到当前用户调用进程链上的上级进程。撤销 Claude/Codex 的桌面主体不能禁用普通终端取这三把 key；关闭这项能力要撤销受信设备或删除厂商登记，已交出的明文不能撤回。现有主体撤销还会推进共享撤销 epoch，使旧受信设备封套暂时返回 `trusted_device_unlock_rebind_required`；这不等于为原生交付建立了按主体的权限隔离。合法重绑受信设备后，该路径仍不检查被撤销的桌面主体。本次未放宽这个 epoch 校验。官方主机检查不按 API 路径限制合法用法；持钥传输默认校验 TLS、不跟随跳转，原生客户端获得 Key 后由客户端负责后续网络行为。

旧 Claude Code DeepSeek 内置 Profile 已退役。Claude 的第三方上下文窗口只接受当前 Profile 显式声明的 modelMetadata；未知或自定义模型不猜测容量，且 AICLI 不设置会提前压缩的 `CLAUDE_AUTOCOMPACT_PCT_OVERRIDE`，也不默认禁用自动/手动压缩。

本机主用入口 `codex-ollama-main`、`claude-ollama-main`、`opencode-ollama-main`、`qwen-code-ollama-main` 统一使用 Qwen3.8 27B 的受管运行标签 `qwen3.8-27b:256k`，保留 262144（256K）最大上下文。Codex 使用 Responses、`max` 和独立受管目录；原显式 27B Profile 保留兼容。运行标签复用官方 `qwen3.8:27b` 的 Q4_K_M 权重，固定 `num_ctx 262144` 与 `draft_num_predict 0`，避免 Ollama 0.33.1 的 MTP 草稿上下文初始化崩溃，不降低模型思考等级。先运行 `scripts\Setup-Qwen38-27B256K.ps1` 安装并回读；它可更新 OpenCode Desktop 的现有 27B 目录项。`codex-ollama-review` 继续使用 Qwen3.6 35B 的独立复核路线。

Codex 的 DeepSeek Key 与 Qwen、GLM 一样按厂商引用密码中心，不在 AICLI 内另存 Profile DPAPI 副本。受管 Codex TOML 只写 `env_key`，不会复制官方手工示例中的明文 `experimental_bearer_token` 或 `preferred_auth_method`。Qwen Code 0.21 与 OpenCode 1.18.8 虽有上游原生 DeepSeek 接入，但 AICLI 当前只为它们提供禁网的 machine-only 沙箱，且尚无隔离真实 Key 的远程 egress relay；因此不提供这两类 DeepSeek Profile。

以后更换本地模型时，由维护者先准备 `data/providers/codex-ollama-main.json` 和新模型的独立 catalog，再在仓库根运行：

```
# 默认只列差异；确认候选配置后同步其他三个引擎的模型身份。
pwsh -NoProfile -File scripts/Sync-LocalModelProfiles.ps1 -Json
pwsh -NoProfile -File scripts/Sync-LocalModelProfiles.ps1 -Apply -Json
```

该命令保留 262144 上下文、各引擎自己的输出/压缩/启动参数，以及原型号专用入口。换型后按输出的 `manual_review` 复核保留的来源链接和说明文字。复核模型独立修改 `codex-ollama-review.json` 与对应目录。不要覆盖旧型号共用的 catalog 或

运行标签。随后预览并应用本项目的统一同步命令，按原有流程安装、运行 `doctor` 并验收实际使用的入口；配置同步成功不表示新模型已通过现场验收。已有受管版本可按 Git 中的原配置回退并重新安装。当前 `Setup-Qwen38-27B256K.ps1` 只负责它命名的 Qwen 版本，不用于创建未来模型。

这些工具供低频、明确选择本地模型时使用。是否调用本地模型由上层 AI 根据任务判断；本地可用不会改变原生子代理的选型，也不会自动替换 Luna Max。

Ollama 公共模板使用默认地址 `127.0.0.1:11434`，不再包含某台机器的私有端口或模型别名。使用前先确认服务和模板所列模型实际存在：

```
ollama list
aicli doctor codex-ollama
```

3.3 配置第三方 API

以 Qwen3.7/3.8 Max 与 DeepSeek 为例：

```
aicli profile configure codex-qwen3-7-max-paygo --reuse-secret-from codex-qwen3-8-max-paygo
aicli profile configure codex-qwen3-8-max-paygo
aicli profile configure codex-deepseek-flash
aicli profile configure codex-deepseek-v4-pro --reuse-secret-from codex-deepseek-flash
```

配置过程中先显示引擎、Provider、套餐和数据去向。Qwen、GLM、DeepSeek 直接保存厂商引用，不询问或复制 Key；保留的其他本地密钥接法仍使用 CurrentUser 存储。秘密不写入 Git、日志、`profile show`、`native` 或 `eject`。

3.4 启动、查看与导出

```
aicli start claude-custom --project "C:\Work\Project"
aicli native claude-custom
aicli eject claude-custom --output .\export-claude-custom
```

- `start` 启动真实上游 CLI。
- `native` 脱敏显示可执行文件、配置、环境差异和数据去向，不执行模型请求。
- `eject` 导出不含 Key/OAuth 的独立配方；目标目录已存在时应换一个新目录。

原生参数必须放在双横线之后：

```
aicli start claude-official -- --effort high
aicli start claude-official -- --permission-mode acceptEdits
aicli start codex-official -- --model gpt-5.6-terra
```

与本工具启动计划冲突的 Provider、认证、`--profile`、`--settings` 或 `-c` 覆盖会被拒绝。不要用透传参数绕过 Profile 的安全边界。

3.5 从 OpenClaw 安全导入（可选）

如果 OpenClaw 已经保存了 DeepSeek 配置，可以用发行包里的导入脚本减少重复录入。它默认只预览，不修改 Profile，也不显示 Key；Qwen3.7 配置会被明确忽略：

```
pwsh -File .\scripts\Import-FromOpenClaw.ps1
```

配置文件不在默认的 `%USERPROFILE%\.openclaw\openclaw.json` 时，显式指定路径：

```
pwsh -File .\scripts\Import-FromOpenClaw.ps1 -OpenClawJson "D:\配置\openclaw.json"
```

确认预览内容后再导入：

```
pwsh -File .\scripts\Import-FromOpenClaw.ps1 -Apply
```

已有同名 Profile 时默认跳过；只有明确要替换时才使用：

```
pwsh -File .\scripts\Import-FromOpenClaw.ps1 -Apply -Force
```

脚本只接受能由 HTTPS 主机名证明身份的 DeepSeek 配置，并要求主机为 `api.deepseek.com`。当前只导入 `codex-deepseek-flash`；V4 Pro、Claude Code、Open Interpreter、Qwen3.7、Qwen Code 与 OpenCode 都不会由该脚本自动创建。未知/OpenAI Base URL 会被拒绝，避免把某家的 Key 误送到另一家。导入的 Key 立即使用 Windows CurrentUser DPAPI 保存；输出、Profile JSON 和日志中都不出现明文。该脚本用于历史环境维护；它产生的旧 DPAPI 副本不会覆盖密码中心的厂商登记，也不是 0.3.19 三家厂商的取密来源。当前版本应直接配置厂商引用；不同 DeepSeek 条目未经确认不合并。需要真实连通证据时再显式执行 Live Test。

4. Doctor 与 Live Test

4.1 Doctor：静态检查，不生成模型内容

```
aicli doctor
aicli doctor codex-deepseek-flash
aicli doctor codex-deepseek-flash --json
```

Doctor 检查 CLI、Profile、秘密引用、端点、有效配置层和本机服务。它不发送模型生成请求，不应消耗模型额度。

状态	含义
通过	单个检查项成功
可用	要求的真实验证和必要能力均有当前证据
可用但有限制	静态就绪但未完成全部 Live Test，或存在已知限制
不可用	关键依赖、配置或测试失败

找不到可选 CLI 不应否定其他引擎；请按 check ID 和“下一步”处理目标 Profile 的问题。

4.2 Live Test：真实请求，可能消耗额度

```
aicli test codex-deepseek-flash --live --level text
aicli test claude-official --live --level text
aicli test claude-official --live --level text --yes
aicli test claude-official --live --level all --yes --json
aicli test codex-ollama-qwen3-8-27b --live --level agent --yes --json
```

`--live` 必须显式给出；没有 `--yes` 时会再次确认。文本测试在随机临时空目录中运行，通过目标 CLI 发出最小请求，并要求正常退出、最终模型正文严格等于 `PONG`，且观测到的工具调用数为 0。Codex 路径仍固定 `danger-full-access`；只要 app-

server 报告首个工具事件就终止并判失败，但这不是执行前工具禁用，工具仍可能在事件被观测前产生本机副作用，只应在明确授权时执行。提示和回复正文不写日志。

层级	含义
`text`	验证目标 CLI、Provider、模型和最小文本链路
`tool`	只验证隔离的单用途 nonce 工具链
`agent`	在全新临时目录完成真实文件任务，并由独立 verifier 验收
`all`	依次验证既有 text+tool 两层；不隐式启动 Agent 任务

无法证明工具隔离时，工具层会跳过并显示“可用但有限制”，不会冒充完整文件或 Shell 编程能力已经通过。

`agent` 只支持 Codex Profile。它复用 root-owned 可恢复 run，瞬态上游、进程或 stream 中断最多自动 exact-resume 3 次；恢复必须保持同一 thread/session、workspace、Profile 指纹、模型、Provider、requested/effective effort 和全访问权限。模型必须真实读取确定性输入并写出结果文件，AICLI 随后用独立 verifier 计算排序、去重、频次、求和和 SHA-256。模型自述“完成”、普通 `PONG`、仅能打开 CLI 或新 thread 重跑都不能通过。回执不保存提示、回复正文、工具载荷、endpoint 或秘密。

当前哪些 Profile 已完成文本验收、哪些仍待验收，以《兼容性与最终验收状态》为准；成功记录也不能跨模型、端点或公开默认配置外推。

5. 当前官方 Rust Open Interpreter

本工具只支持当前官方 Rust Open Interpreter `0.0.21` 或更高版本，不支持旧 Python `open-interpreter 0.4.x`。

5.1 安装和验证

```
irm https://www.openinterpreter.com/install.ps1 | iex
interpreter --version
aicli doctor
```

版本输出必须类似 `interpreter 0.0.21` 或更高。若检测到 `Open Interpreter 0.4.x`，Doctor 会把它识别为不受支持的旧 Python 版本，并给出当前官方安装命令。

5.2 配置和启动

```
aicli start oi-ollama
```

aicli 通过 Rust CLI 的临时 `-c` 配置层写入 Provider、模型、`base_url` 和 `wire_api`。云端 Key 只进入目标子进程环境变量，且明确从 OI Shell 工具环境中排除；不会放进命令行参数。默认不启用跳过审批、跳过沙箱或旧版 `auto_run` 参数。

Open Interpreter 仍具备本机代码执行能力。只在信任的项目目录和 Provider 中使用，并在 OI 自己的 `/permissions` 中确认权限。

官方资料：安装、快速开始、配置、Provider。

6. Claude Code 权限与第三方模型

aicli 默认不替用户设置 Claude permission mode。底栏 `manual mode on` 是 Claude Code 的默认权限档，不是启动失败。

会话内按 `Shift+Tab` 通常在以下模式间循环：

```
manual → acceptEdits → plan
```

查看工具规则：

```
/permissions
```

启动时指定更松的日常写代码模式：

```
aicli start claude-official -- --permission-mode acceptEdits
```

完全绕过权限只适合外部已经隔离、没有不可信网络或文件的环境：

```
aicli start claude-official -- --permission-mode bypassPermissions --dangerously-skip-permissions
```

PowerShell 有时会吞掉参数分隔用的 `--`，导致 `--permission-mode` 等并未传给 Claude。需要透传时优先：

```
pwsh -NoProfile -File $env:LOCALAPPDATA\aicli\bin\aicli.ps1 --% start claude-official -- --permission-mode
acceptEdits
# 或开发仓库：
pwsh -NoProfile -File <安装目录>\bin\aicli.ps1 --% start claude-official -- --permission-mode acceptEdits
```

`bypassPermissions` 必须在新进程启动时启用。第三方模型通常不满足 Claude `auto` 模式的模型或账号条件；这不是 aicli 的故障。

Claude 官方登录与 `ANTHROPIC_API_KEY` 同时存在时，上游可能显示双认证或 connectors disabled 提示。使用第三方 API Profile 时以该 Profile 显示的数据去向为准；使用 `claude-official` 时，先运行 Doctor 检查会抢占官方路径的设置层。

6.1 第三方模型的上下文连续性

原生 ChatGPT + Codex 是基准，保持上游默认。DeepSeek/千问经 Codex 或 Claude Code 时，客户端压缩是可能丢文件细节、数字和未完成分支的摘要；不要为了“省上下文”主动 `/compact`。AICLI 的第三方连续性契约要求一个会话只做一个内聚里程碑，在自然边界把目标与验收、约束/授权/owner、规则与关键文件、改动、决定、测试/Live 缺口、阻塞和下一步写入项目已有 plan/progress/decision 状态；接近窗口时优先 fresh session 或拆任务。压缩后把摘要当线索，重新读取适用的 `AGENTS.md` / `CLAUDE.md`、状态文档以及 `git status` / `git diff`，不建立第二事实源。

本机 27B 主用与显式 27B 兼容入口保持同一模型身份和 262144 context；OpenCode 同时声明 262144 input、32768 output、20000 reserved，并保留最近 4 轮/16384 token。35B 复核与 27B 主用分别验收。改主用模型时须同步实际 Profile、OpenCode 本地入口、Ollama 运行标签及安装后的配置；旧指纹的 Live 记录不会自动继承。

第三方模型（如千问）下，Claude 会话结束页的 **Total cost: $... (costs may be inaccurate due to usage of unknown models) 不是** 阿里云百炼账单。token 量级可能接近真实调用；美元金额多半按 Claude 内置未知模型单价估算，**往往偏高**。真费用以百炼「模型监控 / 账单」为准（调用后约一小时可查）。详见《CLI 中文手册》用量说明。

7. 更新与修复

7.1 只读检查

```
aicli update check
aicli update check codex --json
aicli update guide codex
aicli update guide claude
aicli update guide ollama
aicli update guide interpreter
aicli update guide self
```

`update check` 和 `update guide` 只检查来源、版本并打印同渠道指引，不静默升级 Codex、Claude Code、Ollama 或本工具。识别来源后，应继续使用原安装渠道，避免 npm、WinGet 和原生安装器互相覆盖。

从 `0.3.10` 之前的 AICLI 版本升级时，应使用同一发行包内的 `scripts\Install.ps1`；该安装器会完成旧 Qwen3.7 可验证遗留入口的可恢复隔离，并保留新 06-08 exact Profile。如果预检报告未知或篡改项，安装在写入新版本前中止；请先备份并人工审计，不要绕过门禁。

Open Interpreter Rust 可按上游支持使用：

```
interpreter update
interpreter --version
```

7.2 通用修复顺序

1. `aicli update check` 确认实际来源和版本。
2. 使用同一渠道更新目标 CLI。
3. 运行目标 CLI 的 `--version`。
4. 运行 `aicli doctor <Profile ID>`。
5. 静态通过后，再按需运行显式 Live Test。

8. 卸载、恢复与数据

8.1 普通卸载

```
aicli uninstall
# 非交互确认
aicli uninstall --yes
```

普通卸载会完整移除当前用户模块、`%LOCALAPPDATA%\aicli\bin` 命令垫片、安装器加入的用户 `PATH` 项，以及带产品标记的 PowerShell Profile 自动导入块；不需要再手动清理这些安装集成。

普通卸载默认保留用户 Profile、DPAPI 密钥和导出物，便于以后重装恢复。它不会卸载 Codex、Claude Code、Open Interpreter、Ollama、本地模型，也不会删除上游官方登录。

8.2 彻底清理本工具数据

```
aicli uninstall --purge-user-data --yes
```

`--purge-user-data` 删除本工具的 Profile、DPAPI 密钥、日志和缓存。它不代表远程 OAuth 已撤销。

8.3 数据位置与去向

内容	默认位置
Profile、书签和非秘密设置	`%APPDATA%\AiCliProfileManager`
DPAPI 密文、状态、缓存和日志	`%LOCALAPPDATA%\AiCliProfileManager`
命令垫片	`%LOCALAPPDATA%\aicli\bin`

工具自身默认无遥测。使用某个 Profile 时，提示、项目上下文和相关数据会发送到 `profile show` 显示的 Provider；Ollama 在本机处理，Open Interpreter 还可能在本机执行代码。Live Test 会联系目标 Provider，并可能消耗额度，但不保存提示或回复正文。

9. 按症状排障

找不到 `aicli`

```
$env:Path = "$env:LOCALAPPDATA\aicli\bin;$env:Path"
Import-Module AiCliProfileManager
aicli version
```

仍失败时，新开 PowerShell 7，再用安装包中的 `scripts\Install.ps1 -Force` 修复。

升级后行为仍像旧版 / 报奇怪的 `engine` 属性错误

当前窗口可能仍在用内存中的旧模块：

```
Remove-Module AiCliProfileManager -Force -ErrorAction SilentlyContinue
Import-Module AiCliProfileManager -Force
Get-Module AiCliProfileManager | Format-List Path, Version
aicli version
```

或直接关闭终端重开。判断是否装到预期路径：

```
Get-Command aicli | Format-List *
```

透传给 Claude 的参数像没生效

确认参数写在 `--` 之后；在 PowerShell 中优先使用 `--%`（见上文 §6）。也可用：

```
& aicli @('start','claude-official','--','--permission-mode','acceptEdits')
```

找不到目标 CLI

```
aicli doctor
codex --version
claude --version
interpreter --version
ollama --version
```

只安装你需要的上游 CLI；Open Interpreter 缺失不会影响 Codex 或 Claude Profile。

Open Interpreter 显示旧 Python 0.4.x

旧版不受支持。使用当前官方安装器：

```
irm https://www.openinterpreter.com/install.ps1 | iex
interpreter --version
```

确认输出以小写 `interpreter` 开头且版本不低于 `0.0.21`。

Claude 官方返回 401

这通常表示当前机器的 Claude Code 尚未完成官方登录，不等于 aicli 安装失败。先直接运行 `claude` 完成上游登录，再执行：

```
aicli doctor claude-official
aicli start claude-official
```

官方 Claude 意外走第三方 API

用户、项目或本地 `settings.json` 以及父终端变量可能含 `ANTHROPIC_API_KEY` 或 `ANTHROPIC_BASE_URL`：

```
aicli doctor claude-official
aicli native claude-official
```

Doctor 只报告冲突文件和字段名，不展示值。不要让 aicli 自动改写你的上游设置文件。

Qwen3.7/3.8 exact 或旧 Qwen3.7 Profile 错误

06-08 与 Qwen3.8 各自只使用独立 Workspace 按量入口；其他 Qwen3.7 Max/Plus 身份不提供入口：

```
aicli profile show codex-qwen3-7-max-paygo
aicli profile configure codex-qwen3-7-max-paygo --reuse-secret-from codex-qwen3-8-max-paygo
aicli doctor codex-qwen3-7-max-paygo
aicli profile show codex-qwen3-8-max-paygo
aicli profile configure codex-qwen3-8-max-paygo
aicli doctor codex-qwen3-8-max-paygo
```

Ollama 无法连接或模型不存在

```
ollama list
aicli doctor codex-ollama
```

公共模板使用 `127.0.0.1:11434`。确认 Ollama 正在监听且目标模型已拉取；不要把别人的端口或私有模型别名照搬到公开配置。

Live Test 失败

先运行 Doctor，确认登录、Key、模型和端点。Live Test 的失败或跳过必须保持真实状态，不能用“CLI 能打开”代替连通证据。

10. 完整命令索引

```
aicli
aicli setup
aicli version
aicli help [主题或命令]

aicli profile list [--available] [--json]
aicli profile show <Profile ID> [--json]
aicli profile configure <模板 ID> [--id <新 Profile ID>] [--reuse-existing-secret | --reuse-secret-from
<Profile ID>]
aicli profile set-default <Profile ID>
aicli profile remove <Profile ID> [--yes]

aicli start <Profile ID> [--project <项目路径>] [-- <原生参数...>]
aicli run <Profile ID> --stdin --json --project <项目路径> [--sandbox-policy <policy>] [--no-web-search] [-
- <原生参数...>]
aicli native <Profile ID>
aicli eject <Profile ID> [--output <新目录>]

aicli doctor [Profile ID] [--json]
aicli test <Profile ID> --live [--level text|tool|agent|all] [--yes] [--json]

aicli update check [codex|claude|ollama|interpreter|self] [--json]
aicli update guide [codex|claude|ollama|interpreter|self]
aicli uninstall [--purge-user-data] [--yes]
```

退出码：`0` 成功、`2` 用法错误、`3` 可用但有限制、`4` 不可用、`5` 内部错误、`6` 用户取消。

11. 获取帮助

```
aicli help
aicli help profile
aicli help doctor
aicli help permissions
aicli help compare
```

- CLI 会话操作：《Codex、Claude Code 与 Open Interpreter CLI 中文手册》
- 当前兼容性状态：《兼容性与最终验收状态》
- 安全问题：根目录 `SECURITY.md`
- 隐私说明：根目录 `PRIVACY.md`